

Handwritten Digit Classification (MNIST) with an ANN in Keras

Contents

1. Overview	2
2. The MNIST data	2
3. Scaling the pixels	3
4. The network	4
4.1 Flatten: from an image to a row	4
4.2 The architecture	4
4.3 Counting the parameters	5
5. Compiling and training	6
5.1 The loss for many classes	6
5.2 Training	6
6. Predicting with argmax	7
7. A bigger network and its training curves	7
7.1 The second network	7
7.2 Reading the curves	8
8. Where the network goes wrong	9
9. Predicting a single image	10
10. Summary	11
11. Key terms	11

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the [Course map](#).



- **Builds on:** Standardization ([Note 24](#)); One-hot encoding ([Note 27](#)); Log loss (binary cross entropy) ([Note 73](#)); Confusion matrix ([Note 76](#)); Softmax regression ([Note 79](#)); Overfitting ([Note 91](#)).
- **Leads to:** ANN for regression ([Note 1013](#)); Vanishing gradient ([Note 1018](#)); Early stopping ([Note 1021](#)).
- **Compare with:** K-nearest neighbours ([Note 91](#)); ANN for regression ([Note 1013](#)).

1. Overview

Key point: The same Keras steps as for churn, now with 10 classes: a Flatten layer turns each image into 784 inputs, and an output layer of 10 softmax nodes gives one probability per digit.

The [customer churn Note](#) built a network for **binary** classification: one output node, two classes. Here the task is **multi-class** classification: an image shows one handwritten digit, and the network must say which of the 10 digits, 0 to 9, it is.

The Keras workflow is unchanged: build, compile, fit, predict (Figure 1 of the churn Note). Four things are new:

1. **Input:** an image, not a row of a table; a **Flatten** layer turns it into one row.
2. **Output layer:** 10 nodes with the **softmax** activation.
3. **Loss:** sparse categorical cross-entropy.
4. **Prediction:** the digit with the highest probability, found with **argmax**.

The goal is to see how a network handles more than two classes, not to reach the best accuracy. The Notebook ([notebook.ipynb](#)) runs every step.

2. The MNIST data

Key point: 70,000 images of handwritten digits, 28×28 pixels each, with values 0 to 255. Keras ships them already split: 60,000 for training, 10,000 for testing.

MNIST is a set of 70,000 grey images of handwritten digits, each $28 \times 28 = 784$ pixels (see the [feature engineering Note](#), section 8.1, and the [PCA on MNIST Note](#)). It is so widely used that Keras includes it.

Python: Loading MNIST from Keras.

```
import keras
```

```
(X_train, y_train), (X_test, y_test) = \
```

```
keras.datasets.mnist.load_data()
X_train.shape    # (60000, 28, 28)
X_test.shape     # (10000, 28, 28)
y_train[:3]      # [5 0 4]
load_data() downloads the data once and returns it already split into training and test
sets, so no train_test_split is needed.
```

`X_train` is a 3D array, a 3D tensor (see the [tensors Note](#)): 60,000 images, each a 28×28 grid of numbers. Each number is one pixel's brightness, from 0 (blank) to 255 (full ink), stored row by row. `y_train` holds the **label** of each image: the digit it shows.

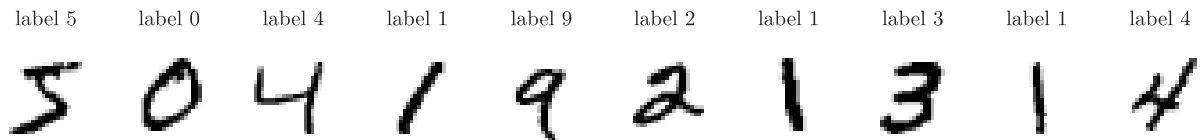


Figure 1: The first 10 training images and their labels

Figure 1 draws the first 10 training images. The first three are a 5, a 0 and a 4, matching `y_train[:3]`.

Python: Drawing one image with Plotly.

```
import plotly.express as px

px.imshow(X_train[0], color_continuous_scale="gray_r")
```

3. Scaling the pixels

Key point: Divide every pixel by 255, the largest possible value, so that all inputs lie between 0 and 1.

As with the churn data, a network trains faster when its inputs share a small range. Pixels already share one range, 0 to 255, so we only shrink it:

1. **In words:** divide each pixel by the largest possible value, 255.
2. **Formula:**

$$x_{\text{scaled}} = \frac{x}{255}$$

3. **Example:** a pixel of 255 becomes $255/255 = 1$, a pixel of 0 stays $0/255 = 0$, and a mid-grey 51 becomes $51/255 = 0.2$.

This is min-max scaling with a known minimum of 0 and maximum of 255 (see the [normalization Note](#)).

Python: Scaling the pixels.

```
X_train = X_train / 255
```

```
X_test = X_test / 255
```

4. The network

Key point: Flatten (784 inputs), a hidden layer of 128 ReLU nodes, an output layer of 10 softmax nodes: 101,770 trainable parameters.

4.1 Flatten: from an image to a row

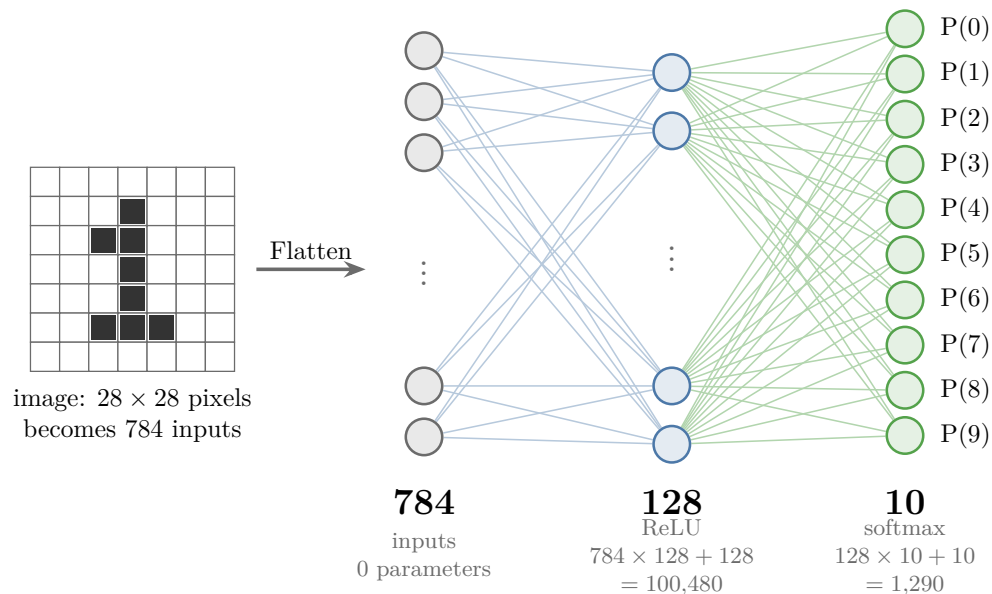
Key point: A Flatten layer lays the 28 rows of pixels side by side, giving one row of 784 numbers.

A Dense layer takes a flat list of numbers, but each image is a 28×28 grid. **Flatten** reshapes any multi-dimensional input into one dimension: first the 28 pixels of row 1, then the 28 of row 2, and so on to row 28, giving $28 \times 28 = 784$ numbers. It is exactly how the [PCA on MNIST Note](#) turns each image into one table row, but done inside the network.

Flatten only rearranges numbers, so it has no weights and nothing to train.

4.2 The architecture

Key point: One input per pixel, 128 hidden nodes, one output node per digit.



101,770 trainable parameters

Figure 2: The first MNIST network: the image is flattened into 784 inputs, which feed 128 ReLU nodes and then 10 softmax nodes, one per digit

Figure 2 shows the network:

- **Input layer:** 784 nodes, one per pixel.
- **Hidden layer:** 128 nodes with the ReLU activation, which usually works best in hidden layers (activation functions get their own Notes later).
- **Output layer:** 10 nodes, one per class.

With more than two classes the output layer has **one node per class**, and the class whose node gives the highest probability is the prediction (see the [MLP intuition Note](#), section 4.3). The 10 nodes use **softmax**, which turns their 10 scores into 10 probabilities that add up to 1 (see the [softmax regression Note](#)). So for every image the network gives $P(0)$, $P(1)$, $\dots$, $P(9)$.

Python: Building the network.

```
from keras import layers
```

```
model = keras.Sequential([
    keras.Input(shape=(28, 28)),          # one image
    layers.Flatten(),                     # 28 x 28 -> 784
    layers.Dense(128, activation="relu"),  # hidden layer
    layers.Dense(10, activation="softmax"), # one node per digit
])
```

The input shape is now (28, 28), one image. The Dense layers work out their own number of inputs from the layer before.

4.3 Counting the parameters

Key point: 100,480 parameters into the hidden layer and 1,290 into the output layer: over 100,000 numbers to learn.

`model.summary()` lists:

Layer	Output shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290
Total		101,770

With the counting rule of the [MLP notation Note](#):

$$784 \times 128 + 128 = 100,480, \quad 128 \times 10 + 10 = 1,290$$

This is the network previewed in the Extra box of section 3 of that Note. Over 100,000 weights and biases is already far more than the 276 of the churn network, though small by deep learning standards.

5. Compiling and training

Key point: Sparse categorical cross-entropy takes the labels as plain digits 0 to 9; categorical cross-entropy needs them one-hot encoded. Otherwise they are the same loss.

5.1 The loss for many classes

Key point: Both losses are categorical cross-entropy; “sparse” only means the labels stay as integers.

With a softmax output, the loss is **categorical cross-entropy**: the average of $-\log$ of the probability the network gives the true class (see the [softmax regression Note](#), section 4.2, and the [perceptron loss Note](#), section 8). Keras offers it in two forms, which differ only in how the labels are written:

Loss	Labels	Label of a “5”
categorical cross-entropy	one-hot vectors	[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
sparse categorical cross-entropy	plain integers	5

Our labels are integers 0 to 9, so the sparse form saves a one-hot encoding step (see the [one-hot encoding Note](#)). The loss values are identical. Loss functions get their own Note later.

Python: Compiling, and the one-hot alternative.

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="adam")

# only needed for categorical_crossentropy:
keras.utils.to_categorical(5, 10) # [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

The optimizer is again Adam; optimizers are taught later.

5.2 Training

Key point: 10 epochs with 20% of the training images held back for validation.

Python: Training.

```
history1 = model.fit(X_train, y_train, epochs=10,
                    validation_split=0.2)
```

Keras trains on 48,000 images and validates on the other 12,000. With batches of 32 that is 1,500 weight updates per epoch (1500/1500 in the output). The training loss falls from 0.280 after the first epoch to 0.014 after the tenth. The validation loss is lowest after epoch 5 (0.088) and has risen to 0.104 by epoch 10: a first sign of overfitting.

6. Predicting with argmax

Key point: `predict` gives 10 probabilities per image; the position of the largest one is the predicted digit.

`model.predict(X_test)` returns an array of shape (10000, 10): for each test image, the probability of each digit. For the first test image, $P(7)$ is almost exactly 1 and the other nine are almost 0.

To turn the 10 probabilities into one digit we take the position of the largest: **argmax**. Since position k holds $P(k)$, the position *is* the digit.

1. **In words:** the predicted class is the index of the largest probability.
2. **Formula:**

$$\hat{y} = \arg \max_k P(k), \quad k = 0, 1, \dots, 9$$

3. **Example:** if an image gets $P(1) = 0.03$, $P(2) = 0.90$, $P(7) = 0.05$ and 0.02 shared among the rest, the largest is at position 2, so $\hat{y} = 2$.

Python: From probabilities to digits.

```
from sklearn.metrics import accuracy_score
```

```
y_prob = model.predict(X_test)      # shape (10000, 10)
y_pred = y_prob.argmax(axis=1)      # [7 2 1 ...]
accuracy_score(y_test, y_pred)      # 0.9763
axis=1 takes the argmax across each row, one answer per image.
```

The first network gets **97.63%** of the 10,000 test images right. For comparison, KNN on all 784 pixels reached 96.8% in the [PCA on MNIST Note](#), and took far longer to predict. A plain network with one hidden layer and no tuning already does better.

Extra: Image data is the home ground of the **convolutional neural network (CNN)**, which looks at small patches of pixels instead of treating every pixel as a separate input. CNNs reach over 99% on MNIST; they are taught in later Notes.

7. A bigger network and its training curves

Key point: An extra hidden layer and 25 epochs make the training accuracy reach 99.7%, but the test accuracy drops slightly to 97.54%: the network overfits.

7.1 The second network

Key point: 784-128-32-10, with accuracy tracked: 104,938 parameters.

To try to improve, we add a second hidden layer of 32 ReLU nodes, train for 25 epochs and track accuracy with `metrics=["accuracy"]` (see the [customer churn Note](#), section 7.3).

$$\underbrace{784 \times 128 + 128}_{100,480} + \underbrace{128 \times 32 + 32}_{4,128} + \underbrace{32 \times 10 + 10}_{330} = 104,938$$

Python: The second network.

```
model2 = keras.Sequential([
    keras.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model2.compile(loss="sparse_categorical_crossentropy",
               optimizer="adam", metrics=["accuracy"])
history = model2.fit(X_train, y_train, epochs=25,
                    validation_split=0.2)
```

The test accuracy is **97.54%**, a little *lower* than the first network's 97.63%. More capacity and more epochs did not help.

7.2 Reading the curves

Key point: The training loss keeps falling while the validation loss climbs after epoch 4: a clear case of overfitting.

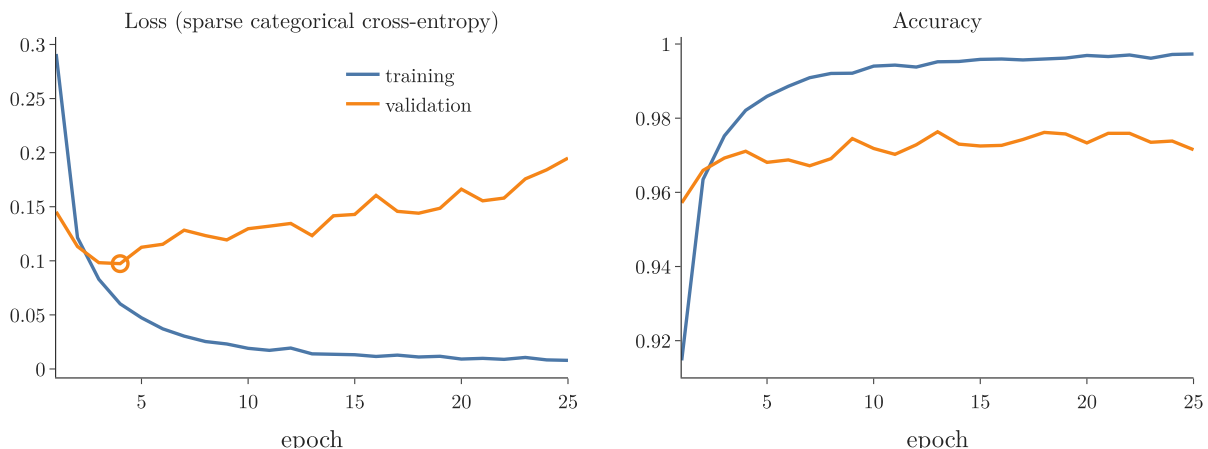


Figure 3: Training curves of the second network. The circle marks the lowest validation loss, after epoch 4.

Figure 3 explains the result:

- **Loss (left):** the training loss falls steadily to 0.008. The validation loss is lowest after epoch 4 (0.097), then rises to 0.195 by epoch 25.

- **Accuracy (right):** training accuracy climbs to 99.7%, validation accuracy stays around 97 to 97.6%.

After epoch 4 the network keeps getting better on the images it trains on and worse on images it has not seen. It is memorising the training set: **overfitting** (see the [bias-variance Note](#)). The churn network showed a small gap; here it is large. Regularization, dropout and stopping training at the right epoch fix this; they come in later Notes.

Extra: If an accuracy curve is requested but `metrics=["accuracy"]` was not given to `compile`, `history.history` has no `accuracy` key and the plot fails with a `KeyError`. The model must be compiled again with the metric and retrained.

8. Where the network goes wrong

Key point: 246 of the 10,000 test images are misclassified; the most common mistake is a 2 read as a 7.

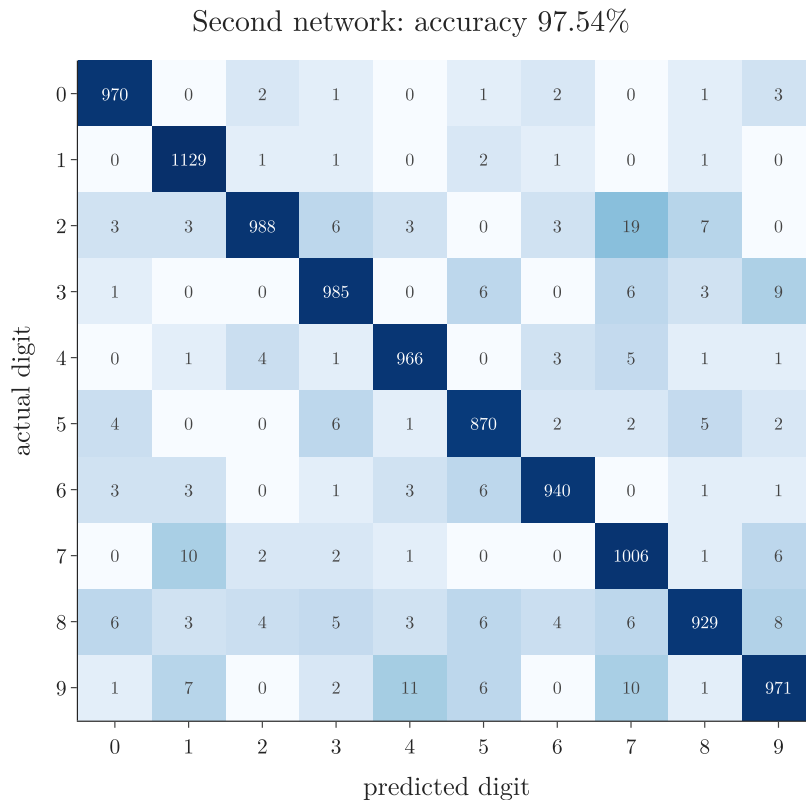


Figure 4: Confusion matrix of the second network on the 10,000 test images. Rows: actual digit; columns: predicted digit.

Figure 4 is the 10×10 confusion matrix (see the [accuracy and confusion matrix Note](#), section 5). The diagonal holds the correct predictions. The largest off-diagonal cells are the most common mistakes:

Actual	Predicted	Count
2	7	19
9	4	11
7	1	10
9	7	10

These pairs look alike when handwritten: a 2 with a straight base resembles a 7, a 9 with an open top resembles a 4. Figure 5 shows five of the misclassified test images; several are hard even for a person.

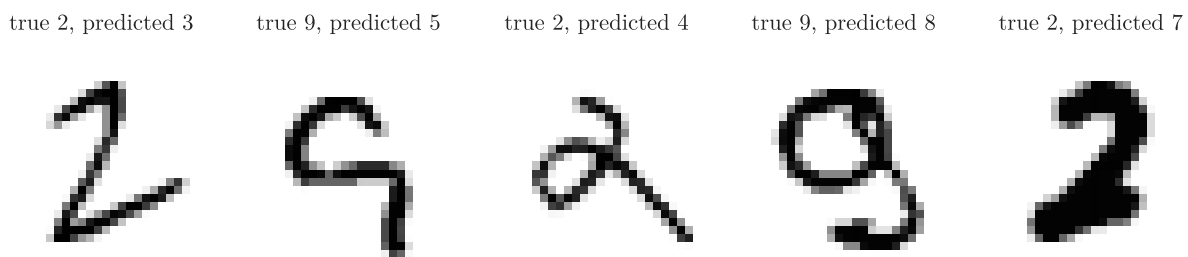


Figure 5: Five test images the second network gets wrong

9. Predicting a single image

Key point: `predict` expects a batch of images, so one image is reshaped from (28, 28) to (1, 28, 28) first.

Keras models always predict a batch of rows. A single image has shape (28, 28); `reshape(1, 28, 28)` turns it into a batch of one.

Python: Predicting one test image.

```
p = model2.predict(X_test[0].reshape(1, 28, 28))
p.argmax(axis=1)      # [7]: the first test image is a 7
```

The first two test images, a 7 and a 2, are both predicted correctly, each with a probability that rounds to 1.

10. Summary

	Churn (binary)	MNIST (multi-class)
Input	11 columns, standardized	28×28 image, divided by 255, flattened
Output layer	1 node, sigmoid	10 nodes, softmax
Loss	binary cross-entropy	sparse categorical cross-entropy
Prediction	probability > 0.5	argmax of 10 probabilities
Test accuracy	86.45%	97.63% (first), 97.54% (second)

- MNIST: 60,000 training and 10,000 test images of 28×28 pixels, loaded with `keras.datasets.mnist.load_data()`.
- Divide pixels by 255; a Flatten layer turns each image into 784 inputs.
- Multi-class output: one softmax node per class; predict with `argmax(axis=1)`.
- Sparse categorical cross-entropy for integer labels, categorical cross-entropy for one-hot labels.
- A bigger network trained longer overfitted: training loss down, validation loss up after epoch 4.

11. Key terms

Term	Meaning
MNIST	70,000 images of handwritten digits, 28×28 pixels, labelled 0 to 9
<code>keras.datasets.mnist</code>	Keras's built-in copy of MNIST, already split 60,000 / 10,000
Label	The true class of an example, here the digit an image shows
Flatten layer	A layer that reshapes a multi-dimensional input into one dimension; no parameters
Softmax output layer	An output layer with one node per class whose outputs are probabilities adding up to 1
Sparse categorical cross-entropy	Categorical cross-entropy for labels written as integers
<code>to_categorical</code>	Keras function that one-hot encodes integer labels
argmax	The position of the largest value; on 10 class probabilities, the predicted class
Convolutional neural network (CNN)	A network built for images that looks at small patches of pixels; taught later